



고객을 세그멘테이션하자 [프로젝트] - 박의진

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
SELECT *  
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`  
LIMIT 10
```

[결과 이미지를 넣어주세요]

쿼리 결과									
작업 정보		결과	시각화	JSON	실행 세부정보	실행 그래프			
행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	
1	536365	85123A	WHITE HANGING HEART T-LIGHT	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom	
2	536365	71293	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
3	536365	84406B	CREAM CUPID HEARTS COAT H.	6	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom	
4	536365	844296	KNITTED UNION FLAG HOT WAL.	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
5	536365	844296	RED WOOLLY HOTTIE WHITE H.	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
6	536365	22782	SET 7 BAMBUSHKA NESTING BO.	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom	
7	536365	21780	GLASS STAR FROSTED T-LIGHT	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom	
8	536365	22833	HANG WARMER UNION JACK	6	2010-12-01 08:26:00 UTC	1.85	17850	United Kingdom	
9	536365	22832	HANG WARMER RED POLKA DOT	6	2010-12-01 08:26:00 UTC	1.85	17850	United Kingdom	
10	536367	84479	ASSORTED COLOUR BIRD ORN.	12	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom	

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT COUNT(*) AS total_rows  
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

작업 정보		결과	시각화	JSON	실행 세부정보	실행 그래프
행	total_rows					
1	541909					

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
SELECT  
COUNT(InvoiceNo) AS cnt_invoice_no,  
COUNT(StockCode) AS cnt_stockcode,  
COUNT(Description) AS cnt_description,  
COUNT(Quantity) AS cnt_quantity,  
COUNT(InvoiceDate) AS cnt_invoicedate,  
COUNT(UnitPrice) AS cnt_unitprice,  
COUNT(CustomerID) AS cnt_customerid,  
COUNT(Country) AS cnt_country  
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

작업 정보

결과

시각화

JSON

실행 세부정보

실행 그래프

행	cnt_invoice_no	cnt_stockcode	cnt_description	cnt_quantity	cnt_invoicedate	cnt_unitprice	cnt_customerid	cnt_country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```
# UNION ALL로 세로형태로 결과값을 볼거임으로 별칭은 통일, 결측치 비율 계산값이 각 컬럼네임 옆에 세로로 짝 이어서 붙을것임.
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'cnt_stockcod' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

UNION ALL
SELECT
    'Country' AS column_name,
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보	결과	시각화	JSON	실행 세부정보	실행 그래프
행	column_name	missing_percenta...			
1	InvoiceNo	0.0			
2	cnt_stockcod	0.0			
3	Description	0.27			
4	Quantity	0.0			
5	InvoiceDate	0.0			
6	UnitPrice	0.0			
7	CustomerID	24.93			
8	Country	0.0			

결측치 처리 전략

- **StockCode = '85123A'의 Description**을 추출하는 쿼리문을 작성하기

```
--일관성 문제를 확인하기 위해 특정 컬럼에 대한 값 조회 StockCode = '85123A' 인 행들의 Description을 전부 출력하는 쿼리
-- DISTINCT로 중복제외한 고유값 중심으로 상황 살피기
SELECT DISTINCT Description
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE StockCode = '85123A';
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보	결과	시각화	JSON	실행 세부정보	실행 그래프
행	Description				
1	WHITE HANGING HEART T-LIGH...				
2	?				
3	wrongly marked carton 22804				
4	CREAM HANGING HEART T-LIG...				

결측치 처리

- **DELETE** 구문을 사용하며, **WHERE** 절을 통해 데이터를 제거할 조건을 제

```
--CustomerID, Description결측치 제거

DELETE FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE CustomerID IS NULL;

DELETE FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE Description IS NULL;
```

[결과 이미지를 넣어주세요]

작업 정보 **결과** 실행 세부정보 실행 그래프

i 이 문으로 data의 행 135,080개가 삭제되었습니다.

← 쿼리 결과

작업 정보 **결과** 실행 세부정보 실행 그래프

i 이 문으로 data의 행 0개가 삭제되었습니다.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

--중복된 행의 수를 세기 (모든 컬럼에 그룹 함수 적용해서 카운트가 1보다 큰 데이터를 세기)

```
SELECT
  COUNT(*) AS duplicated_rows
FROM (
  SELECT
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country,
    COUNT(*) AS cnt
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country
  HAVING cnt > 1
);
```

[결과 이미지를 넣어주세요]

← 쿼리 결과	
작업 정보	결과
시각화	
행	duplicated_rows
1	4837

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기
 - CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

```

--- 중복값이 제외된 데이터셋으로 쓰기
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;

```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보	결과	실행 세부정보	실행 그래프
-------	----	---------	--------

이 문으로 이름이 data인 테이블이 교체되었습니다.

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo의 개수를 출력하기

```

-- 데이터 전처리 (3) 오류값 처리
-- 고유한 인보이스값 보기
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoiceNo
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`

```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보	결과	시각화
-------	----	-----

행	unique_invoiceNo
1	22190

- 고유한 InvoiceNo를 앞에서부터 100개를 출력하기

```

SELECT DISTINCT InvoiceNo
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
LIMIT 100;

```

[결과 이미지를 넣어주세요]

작업 정보	결과	시각화	JSON	실행 세부
명	InvoiceNo			
1	541431			
2	C541433			
3	537626			
4	542237			
5	549222			
6	556201			
7	562032			
8	573511			
9	581180			
10	539318			
11	541998			
12	548955			
13	568172			
14	577609			
15	543037			
16	544156			
17	545323			
18	545332			
19	546869			
20	547390			
21	567505			
22	568699			
23	574275			
24	C545329			
25	C545330			
26	C547388			
27	553900			
28	550911			
29	552449			
30	541430			

- InvoiceNo 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

[결과 이미지를 넣어주세요]

- 구매 건 상태가 Canceled 인 데이터의 비율(%) - 소수점 첫번째 자리까지
- (취소주문수 / 전체주문수) * 100 → round (, 1)

```
-- ROUND(취소주문 수 -> CASE WHEN 조건쓰기 THEN 1 ELSE 0 END / 전체 주문수 * 100 , 1)
SELECT
ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100, 1) AS cancel_rate
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

쿼리 결과	
작업 정보	결과
명	cancel_rate
1	2.2

StockCode 살펴보기

- 고유한 StockCode 의 개수를 출력하기

```
-- 고유한 스톡코드 보기
SELECT COUNT(DISTINCT StockCode) AS unique_StockCode
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]



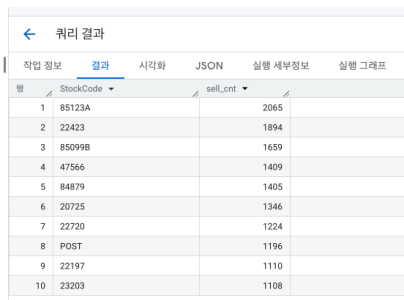
```
-- 153 --> 고유한 스톡코드 찾기
154 SELECT COUNT(DISTINCT StockCode) AS unique_StockCode
155 FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
156
```

unique_StockCode
3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기

```
-- 상위 10개 (그룹바이로 묶어주고 그 그룹별 행개수 카운트하면 고유값별 빈도수 카운트됨 이걸 오더바이로 DESC)
SELECT
  StockCode, COUNT(*) AS sell_cnt
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

[결과 이미지를 넣어주세요]



StockCode	sell_cnt
85123A	2065
22423	1894
85099B	1659
47566	1409
84879	1405
20725	1346
22720	1224
POST	1196
22197	1110
23203	1108

- StockCode** 의 컬럼에 있던 값 중에서 (정규식) 숫자를 빈칸으로 치환해서 문자만 남기고 원래 코드문자에서 처리한 문자를 빼서 숫자 개수를 셈.
 - 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
--스톡코드 문자열이 어떤식으로 되어있는지 확인해보기
---(숫자를 빈칸으로 대체한 뒤 원래 문자수에서 처리한 문자수를 빼서 숫자개수를 계산)
WITH UniqueStockCodes AS (
  SELECT DISTINCT StockCode
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
)
SELECT
  LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count,
  COUNT(*) AS stock_cnt
FROM UniqueStockCodes
GROUP BY number_count
ORDER BY stock_cnt DESC;

--- 숫자가 0-1개인 값들에는 어떤 코드가 들어가있나?
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
)
WHERE number_count IN(0,1);
```

[결과 이미지를 넣어주세요]

← 쿼리 결과		
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프		
행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세어 숫자 개수 계산한 뒤
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```
SELECT
  ROUND(SUM(CASE WHEN (LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', ''))) <= 1 THEN 1 ELSE 0 END)
    / COUNT(*) * 100, 2) AS abnormal_stockcode
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

주문형 처리 할당량 사용 중		
← 쿼리 결과		
작업 정보 결과 시각화		
행	abnormal_stockcode	
1	0.48	

- 제품과 관련되지 않은 거래 기록을 제거하기 (조심할것.. 데이터 날라가면 다시 원복해서 해라

```
--위에 해당 코드는 데이터 안에서 삭제하기
DELETE FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
      LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  )
  WHERE number_count IN (0, 1)
);
```

[결과 이미지를 넣어주세요]

← 쿼리 결과		
작업 정보 결과 실행 세부정보 실행 그래프		
이 문으로 data의 행 1,915개가 삭제되었습니다.		

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
---이번에 description 살펴보기
--- 고유한 디스크립션별 출현 빈도를 계산하고 상위 30개 출력
SELECT Description, COUNT(*) AS description_cnt
```

```
FROM `project-5278bde0-2369-4e6d-8e7`.`modulabs_project`.`data`
GROUP BY Description
ORDER BY COUNT(*) DESC
LIMIT 30;
```

[결과 이미지를 넣어주세요]

행	Description	description_cnt
1	WHITE HANGING HEART T-LIGHT...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORN...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062
10	SPOTTY BUNTING	1026
11	PAPER CHAIN KIT 50'S CHRIST...	1013
12	LUNCH BAG SPACEBOY DESIGN	1006
13	LUNCH BAG CARS BLUE	1000
14	HEART OF WICKER SMALL	990
15	NATURAL SLATE HEART CHALK...	989
16	JAM MAKING SET WITH JARS	966
17	LUNCH BAG PINK POLKADOT	961
18	LUNCH BAG SUKI DESIGN	922
19	ALARM CLOCK BAKELIKE RED	917
20	WOODEN PICTURE FRAME WHI...	900
21	REX CASH+CARRY JUMBO SHO...	900
22	JUMBO BAG PINK POLKADOT	897
23	SET OF 4 PANTRY JELLY MOUL...	890
24	LUNCH BAG APPLE DESIGN	890
25	BAKING SET 9 PIECE RETROSP...	885
26	RECIPE BOX PANTRY YELLOW D...	883
27	JAM MAKING SET PRINTED	883
28	LUNCH BAG WOODLAND	850
29	ROSES REGENCY TEACUP AND ...	844
30	VICTORIAN GLASS HANGING T...	843

• 서비스 관련 정보를 포함하는 행들을 제거하기

```
-- 서비스 관련 정보를 포함하는 행들 제거하는 쿼리문
DELETE FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE
REGEXP_CONTAINS(Description, r'(?i)(CARRIAGE|POSTAGE|SAMPLES|MANUAL|IMAGE|PACKING|CHARGES)'); -- 대소문자구
분없이(?i)
```

[결과 이미지를 넣어주세요]

작업 정보	결과	실행 세부정보	실행 그래프
이 문으로 data의 행 670개가 삭제되었습니다.			

• 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기 (기존 Description 삭제하고 수정된거를 추가)

```
-- 대소문자를 대문자로 통일한 뒤 그 수정된 컬럼으로 얹어쓰기
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.data` AS
SELECT
* EXCEPT (Description),
UPPER(Description) AS Description
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

작업 정보	결과	실행 세부정보	실행 그래프
이 문으로 이름이 data인 테이블이 교체되었습니다.			

UnitPrice 살펴보기

- UnitPrice 의 최솟값, 최댓값, 평균을 구하기

```
---UnitPrice의 최솟값, 최댓값, 평균을 구해 보세요.
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보

결과

시각화

JSON

실행 세부정보

실행 그래프

행	min_price	max_price	avg_price
1	0.0	649.5	2.906593821346...

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기

```
SELECT
  COUNT(*) AS zero_price_cnt,
  MIN(Quantity) AS min_qty,
  MAX(Quantity) AS max_qty,
  AVG(Quantity) AS avg_qty
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE UnitPrice = 0;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보		결과	시각화	JSON	실행 세부정보	실행 그래프
행	zero_price_cnt	min_qty	max_qty	avg_qty		
1	33	1	12540	420.5151515151...		

- UnitPrice = 0 를 제거하고 일관된 데이터셋을 유지하기

```
-- 결국 이상한 단가 0 행들은 제거해서 정상 데이터셋만 유지하도록 얹어치기 (0이상만 남기기.. 음수는 환불 취소 거래였음. 0은 이상한 오류 값이라고 판단)
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.data` AS
SELECT *
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE UnitPrice > 0;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보

결과

실행 세부정보

실행 그래프

이 문으로 이름이 data인 테이블이 교체되었습니다.

11-7. RFM 스코어

Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```
-----RFM 분석으로 넘어가기
--- 1. Recency 부분
--- 인보이스데이트부터 연월일 자료형으로 변경 (DATE)
SELECT
  DATE(InvoiceDate) AS InvoiceDay
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	InvoiceDay
1	2011-01-18
2	2011-01-18
3	2010-12-07
4	2010-12-07
5	2010-12-07
6	2010-12-07
7	2010-12-07

- 가장 최근 구매 일자를 **MAX()** 함수로 찾아보기

```
--- 윈도우함수를 써서 최근구매일 추가해줘라.
SELECT
  MAX(DATE(InvoiceDate)) OVER() AS most_recent_date,
  DATE(InvoiceDate) AS InvoiceDay,
  *
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

쿼리 결과									
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프									
행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	UnitPrice	Custor
1	2011-12-09	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04		
2	2011-12-09	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04		
3	2011-12-09	2010-12-07	537626	22726	4	2010-12-07 14:57:00 UTC	3.75		
4	2011-12-09	2010-12-07	537626	21171	12	2010-12-07 14:57:00 UTC	1.45		
5	2011-12-09	2010-12-07	537626	851678	30	2010-12-07 14:57:00 UTC	1.25		
6	2011-12-09	2010-12-07	537626	849970	6	2010-12-07 14:57:00 UTC	3.75		
7	2011-12-09	2010-12-07	537626	85116	12	2010-12-07 14:57:00 UTC	2.1		
8	2011-12-09	2010-12-07	537626	22728	4	2010-12-07 14:57:00 UTC	3.75		

- 유저 별로 가장 큰 **InvoiceDay**를 찾아서 가장 최근 구매일로 저장하기

```
-- 유저별로 최근 구매일을 찾아보자
SELECT
  CustomerID,
  MAX(DATE(InvoiceDate)) AS most_recent_date
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
GROUP BY CustomerID
ORDER BY most_recent_date DESC;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보	결과	시각화	JSON	실행 세부정보	실행 그래프
행	CustomerID	most_recent_date			
1	13069	2011-12-09			
2	12526	2011-12-09			
3	17389	2011-12-09			
4	17364	2011-12-09			
5	12518	2011-12-09			
6	18102	2011-12-09			
7	12985	2011-12-09			
8	14422	2011-12-09			
9	15779	2011-12-09			

- 가장 최근 일자(**most_recent_date**)와 유저별 마지막 구매일(**InvoiceDay**)간의 차이를 계산하기

(EXTRACT 대신 DATE_DIFF 로 구한값 선택)

```
-- 가장 최근일자 빼기 유저별 마지막 구매일 간의 차이 구하기
SELECT
  CustomerID,
  DATE_DIFF(MAX(InvoiceDay) OVER(), InvoiceDay, DAY) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(Date(InvoiceDate)) AS InvoiceDay
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
)
ORDER BY recency;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보	결과	시각화	JSON	실행
행	CustomerID	recency		
1	12346	325		
2	12347	2		
3	12348	75		
4	12349	18		
5	12350	310		
6	12352	36		
7	12353	204		
8	12354	232		
9	12355	214		

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 **user_r** 이라는 이름의 테이블로 저장하기

```
-- Recency 계산한것은 user_r 테이블로 저장하시오

CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_r` AS
SELECT
  CustomerID,
  DATE_DIFF(
    MAX(InvoiceDay) OVER(), -- 전체 데이터 중 가장 최근 구매일
    InvoiceDay,             -- 고객별 마지막 구매일
    DAY
  ) AS Recency
FROM (
  SELECT
    CustomerID,
    MAX(Date(InvoiceDate)) AS InvoiceDay
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

← 쿼리 결과			
작업 정보	결과	실행 세부정보	실행 그래프
❗ 이 문으로 이름이 user_rf인 새 테이블이 생성되었습니다.			

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```
---1. 전체 거래 건수 계산 (고객이 몇 번 결제 했나)
SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
GROUP BY CustomerID ;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과			
작업 정보	결과	시각화	JSON
행	CustomerID	purchase_cnt	
1	12346	2	
2	12347	7	
3	12348	4	
4	12349	1	
5	12350	1	
6	12352	8	
7	12353	1	
8	12354	1	
9	12355	1	
10	12356	3	
11	12357	1	
12	12358	2	
13	12359	6	
14	12360	3	
15	12361	1	
16	12362	13	
17	12363	2	
18	12364	4	
19	12365	1	
20	12367	1	
21	12370	4	

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
---2. 구매한 아이템의 총 수량 계산
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
GROUP BY CustomerID;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과			
작업 정보	결과	시각화	JSON
행	CustomerID	item_cnt	
1	12346	0	
2	12347	2408	
3	12348	2332	
4	12349	630	
5	12350	196	
6	12352	463	
7	12353	20	
8	12354	530	
9	12355	240	
10	12356	1573	
11	12357	2708	
12	12358	242	
13	12359	1099	
14	12360	1156	
15	12361	90	
16	12362	2180	
17	12363	408	
18	12364	1489	
19	12365	173	
20	12367	172	
21	12370	2349	

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 **user_rf** 라는 이름의 테이블에 저장하기

```

---3. 위의 값들도 추가해서 앞에 테이블에 합치기
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_rf` AS

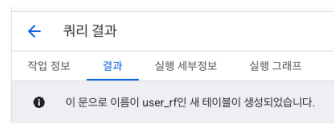
-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.Recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID;

```

[결과 이미지를 넣어주세요]



Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```

----MONETARY 단계
-- 총 주문금액 구하기
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
GROUP BY CustomerID;

```

[결과 이미지를 넣어주세요]

← 쿼리 결과			
작업 정보		결과	시각화
		결과	JSON
행	CustomerID	user_total	avg
1	12346	0.0	
2	12347	4310.0	
3	12348	1437.2	
4	12349	1457.5	
5	12350	294.4	
6	12352	1265.6	
7	12353	80.0	
8	12354	1078.6	
9	12355	459.4	
10	12356	2487.4	
11	12357	6207.7	
12	12358	928.1	
13	12359	6183.0	
14	12360	2302.1	
15	12361	174.9	
16	12362	4665.6	
17	12363	552.0	
18	12364	1208.1	
19	12365	320.7	
20	12367	150.9	
21	12370	9421.9	

• 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt`로 나누어서 3) `user_rfm` 테이블로 저장하기

```

---(2) 평균 주문금액 구해서 위에서 만든 RF 테이블에 그 값도 붙여준 테이블 rfm 만들기
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_average
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_rf` rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;

```

[결과 이미지를 넣어주세요]

← 쿼리 결과			
작업 정보	결과	실행 세부정보	실행 그래프
 이 문으로 이름이 user_rfm인 새 테이블이 생성되었습니다.			

RFM 통합 테이블 출력하기

• 최종 `user_rfm` 테이블을 출력하기

```

--- 최종 테이블 출력
SELECT *
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_rfm`
ORDER BY CustomerID
LIMIT 100

```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보 결과 실행 세부정보 실행 그래프

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12346	2	0	325	0.0	0.0
2	12347	7	2458	2	4310.0	615.7
3	12348	4	2332	75	1437.2	359.3
4	12349	1	630	18	1457.5	1457.5
5	12350	1	196	310	294.4	294.4
6	12352	8	463	36	1265.4	158.2
7	12353	1	20	204	89.0	89.0
8	12354	1	530	232	1079.4	1079.4
9	12355	1	240	214	459.4	459.4
10	12356	3	1573	22	2487.4	829.1
11	12357	1	2708	33	6207.7	6207.7
12	12358	2	242	1	928.1	464.1
13	12359	6	1599	7	6183.0	1030.5
14	12360	3	1156	52	2302.1	767.4
15	12361	1	90	287	174.9	174.9
16	12362	13	2180	3	4665.6	358.9
17	12363	2	408	109	552.0	276.0
18	12364	4	1499	7	1208.1	302.0
19	12365	1	173	291	320.7	320.7
20	12367	1	172	4	150.9	150.9
21	12370	4	2349	51	3421.9	855.5
22	12371	1	582	99	1528.0	1528.0
23	12372	3	788	71	1196.0	398.7
24	12373	1	196	311	324.6	324.6

페이지당 결과 수: 25

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) `user_rfm` 테이블과 결과를 합치기
- 3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
)
SELECT
  ur.*,
  up.* EXCEPT (CustomerID)
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
  ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]

← 쿼리 결과

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 이름이 user_data인 새 데이터베이스 생성되었습니다.

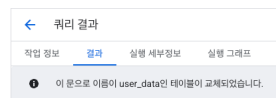
-- (2) 평균구매소요일수를 계산해서 그 결과를 `user_data`에 통합하기

```
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE
      WHEN AVG(interval_) IS NULL THEN 0
      ELSE ROUND(AVG(interval_), 2)
    END AS average_interval
  FROM (
```

```
-- (1) 구매와 구매 사이에 소요된 일수
SELECT
  CustomerID,
  DATE_DIFF(
    DATE(InvoiceDate),
    DATE(LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate)),
    DAY
  ) AS interval_
FROM
  `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
WHERE CustomerID IS NOT NULL
)
GROUP BY CustomerID
)

SELECT
  u.*,
  pi.average_interval
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

[결과 이미지를 넣어주세요]



스키마	세부정보	미리보기	테이블 탐색기	프러뷰	통계	계보	데이터 프로필	데이터 품질
행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inter...
1	14705	1	100	198	179.0	179.0	1	0.0
2	12814	1	48	101	85.9	85.9	1	0.0
3	13841	1	100	252	85.0	85.0	1	0.0
4	13185	1	12	267	71.4	71.4	1	0.0
5	17923	1	50	282	207.5	207.5	1	0.0
6	14679	1	-1	371	-2.5	-2.5	1	0.0
7	14424	1	48	17	322.1	322.1	1	0.0
8	16737	1	288	53	417.6	417.6	1	0.0
9	15753	1	144	304	79.2	79.2	1	0.0
10	15316	1	100	326	165.0	165.0	1	0.0
11	13829	1	-12	359	-102.0	-102.0	1	0.0
12	14576	1	12	372	35.4	35.4	1	0.0
13	16061	1	-1	269	-29.9	-29.9	1	0.0
14	17752	1	192	359	80.6	80.6	1	0.0
15	12791	1	96	373	177.6	177.6	1	0.0
16	13747	1	8	373	79.6	79.6	1	0.0
17	13366	1	144	50	56.2	56.2	1	0.0
18	15400	1	100	20	26.0	26.0	1	0.0

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기
 - 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
 - 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
 - 취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기
(취소 비율은 소수점 두번째 자리)

```
--- 취소 빈도와 취소 비율을 계산해서 또 user_data에 통합해주기
---전체 거래 인보이스 수 (total_transactions)
---cancel_frequency 취소 거래수
```

```

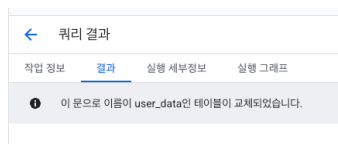
--- cancel_rate 취소 비율 까지 추가해줄거임.
CREATE OR REPLACE TABLE `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_data` AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT IF(InvoiceNo LIKE 'C%', InvoiceNo, NULL)) AS cancel_frequency
  FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.data`
  GROUP BY CustomerID
)

SELECT
  u.*,
  t.* EXCEPT(CustomerID),
  ROUND(t.cancel_frequency / t.total_transactions, 2) AS cancel_rate
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;

```

[결과 이미지를 넣어주세요]



- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 **user_data**를 출력하기

```

--- 출력
SELECT *
FROM `project-5278bde0-2369-4e6d-8e7.modulabs_project.user_data`
ORDER BY CustomerID
LIMIT 100;

```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보

결과

시각화

JSON

실행 세부정보

실행 그래프

명	CustomerID	purchase_cnt	item_cnt	recovery	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	12346	2	0	325	0.0	0.0	1	0.0	2	1	0.5
2	12347	7	2458	2	4310.0	615.7	103	2.02	7	0	0.0
3	12348	4	2332	75	1437.2	359.3	21	10.88	4	0	0.0
4	12349	1	630	18	1457.6	1457.6	72	0.0	1	0	0.0
5	12350	1	196	310	294.4	294.4	16	0.0	1	0	0.0
6	12352	8	463	36	1265.4	158.2	57	3.13	8	1	0.13
7	12353	1	20	204	89.0	89.0	4	0.0	1	0	0.0
8	12354	1	530	232	1079.4	1079.4	58	0.0	1	0	0.0
9	12355	1	240	214	499.4	499.4	13	0.0	1	0	0.0
10	12356	3	1573	22	2487.4	829.1	52	5.32	3	0	0.0
11	12357	1	2708	33	6207.7	6207.7	131	0.0	1	0	0.0
12	12358	2	242	1	928.1	464.1	12	9.31	2	0	0.0
13	12359	6	1599	7	6183.0	1030.5	214	1.3	6	2	0.33
14	12360	3	1156	52	2302.1	767.4	104	1.18	3	0	0.0
15	12361	1	90	287	174.9	174.9	9	0.0	1	0	0.0

회고

[회고 내용을 작성해주세요]

Keep :

- SQL 기본 구조 (SELECT-FROM-WHERE-GROUP BY-ORDER BY) 반복 연습을 통해서 단순 쿼리문은 잘 작성할 수 있는 수준으로 올라선 느낌이다.
- 데이터 전처리 과정을 일단 오류내지 않고 자연스럽게 따라갈 수 있었음. (결측치 처리, 중복 확인, 오류값 확인, 정제, 피쳐 생성까지의 전처리 과정이 다시 한번 머릿속에 구조화됨)
- 정규식(REGEXP), CASE WHEN, LAG, NTILE 같은 함수들이 익숙치 않았으나, 직접 여러 번 써보며 “어떻게 쓰는지”뿐만 아니라 “왜 쓰는지” 감각까지 조금씩 잡혀가는 것 같다.

- SQL이라는 언어를 통해 전처리 과정을 다시 수행해 봄으로써, 실제 현업에서 받을 수 있는 주문고객데이터를 어떻게 비즈니스에 유용하게 전처리 할 수 있는지를 경험하여 레벨업할 수 있는 하루였다.

Problem :

- 정규식을 써야할 때는 나중에 혼자서 필요 여부를 판단하고 어떻게 찾고 적용할 지 알 수 있을까? 걱정이 됨.
- WITH + JOIN + CREATE AS SELECT가 한 번에 등장하면 문법 흐름이 뒤섞여 헷갈림.
- 임시 저장과 실제 저장 사이의 감각이 완전히 체화되지 않았음 (덮어쓰기하는 흐름이 오늘 많아서 갑자기 또 막 헷갈림 → 데이터 테이블을 멀티탭으로 띄워놓고 하는 게 도움이 되는 듯함)
- DATE_DIFF랑 EXTRACT가 왜 Recency 계산에서는 다르게 작동하는지 의문이 생겨서 여러가지로 찾아봤는데.. 일단 달력날짜 기준으로 계산할 때는 DATE_DIFF가 더 정확한 값이라는 것 외에는 EXTRACT 접근이 왜 빅쿼리에서 잘못계산될 수도 있다는 건지 완전하게 이해가 가지 않았음.

Try :

- 복잡한 SQL 쿼리를 작성할 때, 큰 흐름을 3단계로 나누어 보는 습관 만들기

CTE로 중간 결과 만들기 → JOIN으로 묶기 → CREATE TABLE로 저장

- 테이블 생성 또는 덮어쓰기 한 다음에는 테이블 미리보기 띄워놓고 같이 보면서 확인하는 습관들이기. (조회)